

APP-5258 · Validación biométrica Facephi — Identity Validation V2

Versión definitiva · 10 de agosto de 2026

Microservicio: `onboarding-micro-person` · **Proyecto:** `src/micro-person-api`

Stack: .NET 8 · MediatR · FluentValidation · Clean Architecture

Endpoint expuesto: `POST /facephi/validate`

Endpoint integrado: `POST /onboarding/v2/identity` (Facephi Identity API)

Épica: APP-5222 · **Bloquea:** APP-5223 — Enrolamiento de Soft Token mediante validación biométrica Facephi

Cómo usar este documento. Las secciones 1 a 4 son el contexto y el contrato: léelas una vez. La **sección 5 es la guía de trabajo**: nueve cambios, cada uno con su ruta de archivo exacta, el punto donde va y el código. La sección 6 dice en qué orden aplicarlos para que el proyecto compile en cada paso.

Advertencia: el código de la sección 5 se escribió a partir de la revisión del código actual, sin acceso al repositorio ni al SDK de .NET, y **no ha sido compilado**. Los *namespaces* y las rutas se tomaron de los archivos revisados; ajusta lo que difiera.

Índice

1. Contexto y decisiones cerradas
 2. Contrato de Facephi — Identity Validation V2
 3. Tablas de códigos de resultado
 4. Estado actual de la implementación
 5. **Guía de cambios, archivo por archivo**
 6. Orden de aplicación y verificación
 7. Checklist de cierre
 8. Preguntas abiertas
-

1. Contexto y decisiones cerradas

Facephi ya estaba integrado en `onboarding-micro-person` para el proceso de onboarding. APP-5258 **no crea un microservicio ni un flujo paralelo**: añade dentro del mismo micro la operación de validación biométrica que consumirá el enrolamiento de Soft Token (APP-5223).

Decisión	Resultado
¿Endpoints propios de Soft Token?	No. Los endpoints son genéricos de Facephi. Nada de rutas, carpetas ni clases con "softtoken" en el nombre
¿Qué se integra?	<code>POST /onboarding/v2/identity</code> — Identity Validation V2
¿Datos mínimos?	Los que recibe ese endpoint: <code>token1</code> , <code>bestImageToken</code> , <code>method</code> y opcionalmente <code>tracking</code>
¿ requestId ?	No es relevante para nosotros
¿Persistencia en Mongo?	Descartada. La operación es sin estado: entra la captura, sale el veredicto
¿ token2 (dorso del documento)?	No aplica. Identity V2 no tiene ese campo. El modelo del ticket debe corregirse
¿Identity V2 o Authenticate User V2?	Identity V2 , confirmado: el cliente escanea su cédula nuevamente al activar el token

1.1 Por qué Identity V2 y no Authenticate User V2

`authenticateUser/v2` compara el `bestImageToken` del momento contra un **template biométrico almacenado** (`registeredTemplateRaw`) durante el onboarding. Evita volver a escanear el documento, pero obliga a persistir biometría por usuario.

Identity V2 compara **la foto del documento capturada en el momento** contra el selfie. Como el flujo de Soft Token sí vuelve a pedir la cédula al cliente, Identity V2 es el correcto y **no hay que almacenar nada**. Esta decisión elimina de raíz el requisito de la base de datos nueva.

2. Contrato de Facephi — Identity Validation V2

```
POST {FACEPHI_IDENTITY_API_BASE_URL}/onboarding/v2/identity
Content-Type: application/json
```

2.1 Headers

Header	Requerido	Valor
x-api-key	Sí	API key de autorización
family	Sí cuando se envía tracking	OnBoarding

La documentación en español escribe OnBoarding y la inglesa Onboarding . Verificar cuál acepta el ambiente.

2.2 Request

Campo	Tipo	Requerido	Descripción
token1	string	Sí	Imagen de referencia para la comparación facial: base64 abierto o TokenFaceImage , según el método
bestImageToken	string	Sí	bestImage tokenizada generada por el widget Selphi
method	string	Sí	Método de comparación: "3" o "5"
tracking	object	No	Información de seguimiento
tracking.extraData	string	No	Token generado por el SDK Mobile/Web
tracking.operationId	string	No	Identificador de operación del SDK (UUID)

Especificación de method

Método	Qué se envía en token1	Requiere
"3"	Imagen abierta en base64 del frente de la cédula, donde está el rostro	—
"5"	Token del recorte de la foto del documento (TokenFaceImage)	Widget SelphiD Mobile

```
{
  "token1": "/9j/4AAQSkZJRgABAQAAASAB...",
  "bestImageToken": "BAMBAQLNHJoWGPj...",
  "method": "3",
  "tracking": {
    "extraData": "BQABAQG2gBNjuHN4kLmPqYf7R...",
    "operationId": "123e4567-e89b-12d3-a456-426614174000"
  }
}
```

2.3 Response 200

Campo	Tipo	Descripción
serviceTransactionId	string	Identificador de la transacción en la API
serviceResultCode	integer	Resultado de la ejecución del servicio , no del veredicto biométrico
serviceResultLog	string	Detalle de la ejecución
serviceTime	string	Tiempo de procesamiento en milisegundos
facialAuthenticationResult	integer	Resultado de la coincidencia facial
facialAuthenticationLog	string	Detalle de la coincidencia facial
facialAuthenticationSimilarity	number	Similitud facial. 1.0 = 100 %
passiveLivenessResult	integer	Resultado de la prueba de vida pasiva
passiveLivenessLog	string	Detalle de la prueba de vida

```
{
  "serviceTransactionId": "2db602ee-3564-4304-af95-92a52eaae12d",
  "serviceResultCode": 0,
  "serviceResultLog": "[identity] Service executed ok",
  "serviceTime": "2235",
  "facialAuthenticationResult": 3,
  "facialAuthenticationLog": "Positive",
  "facialAuthenticationSimilarity": 0.99214232,
  "passiveLivenessResult": 3,
  "passiveLivenessLog": "Live"
}
```

Identity V2 rompe la convención de los demás endpoints de Facephi. `authenticateUser/v2` y `evaluatePassiveLiveness` usan `serviceFacialAuthenticationResult` y `serviceLivenessResult`; Identity V2 usa `facialAuthenticationResult` y `passiveLivenessResult`, sin prefijo. No copiar modelos entre endpoints.

2.4 Errores HTTP de Facephi

Código	Significado
400	Request inválido — nuestro payload está mal armado
401	API key ausente o inválida
403	La API key no tiene permiso sobre el recurso
502	Facephi devolvió una respuesta inválida
504	Timeout del endpoint

3. Tablas de códigos de resultado

3.1 `serviceResultCode`

Valor	Descripción	HTTP
0	La ejecución del servicio fue exitosa: el módulo procesó la solicitud correctamente	200

Crítico: `serviceResultCode = 0` significa "el servicio corrió bien", **no** "la persona fue aprobada". En la documentación de `evaluatePassiveLiveness`, un rechazo por prueba de vida devuelve `serviceResultCode: 0` con `serviceResultLog: "NoLive"`. El veredicto está en los otros dos campos.

3.2 `facialAuthenticationResult` — coincidencia facial

Código	Resultado	Interpretación
0	NONE	No se pudo realizar la verificación → reintentable
1	NEGATIVE	Los rostros no coinciden → rechazo
3	POSITIVE	Los rostros coinciden → único valor aprobado
4	NONE BECAUSE POSE EXCEED	Posición del rostro inválida → reintentable
5	NONE BECAUSE INVALID EXTRACTIONS	No se pudo extraer el patrón facial → reintentable

3.3 `passiveLivenessResult` — prueba de vida

Código	Resultado	Interpretación
3	Live	Sujeto vivo → único valor aprobado
17	NoLive	No se detectó vida → rechazo
1	Spoof	Deprecado, usar NoLive → rechazo
0	None	No se pudo evaluar → reintentable
2	Uncertain	Deprecado → reintentable
4	NoneBecauseBadQuality	Mala calidad de imagen → reintentable
5	NoneBecauseFaceTooClose	Rostro muy cerca de los bordes → reintentable
6	NoneBecauseFaceNotFound	No se detectaron rostros → reintentable
7	NoneBecauseFaceTooSmall	Rostro muy pequeño → reintentable
8	NoneBecauseAngleTooLarge	Ángulo excede el límite → reintentable
9	NoneBecauseImageDataError	Error de formato de imagen → reintentable
10	NoneBecauseInternalError	Error interno de Facephi → error técnico
11	NoneBecauseImagePreprocessError	Error de preprocesamiento → reintentable
12	NoneBecauseTooManyFaces	Demasiados rostros → reintentable
13	NoneBecauseFaceTooCloseToBorder	Rostro muy cerca del borde → reintentable
14	NoneBecauseFaceCropped	Rostro recortado → reintentable
15	NoneBecauseLicenseError	Error de licencia → error técnico
16	NoneBecauseFaceOccluded	Rostro ocluido → reintentable
18	NoneBecauseEyesClosed	Ojos cerrados → reintentable

Por qué la distinción importa

Un cliente con mala luz (4 , 16 , 18) **no es un fraude**: debe poder recapturar. Un 1 en coincidencia facial o un 17 en prueba de vida sí son rechazos que deben cortar la activación del Soft Token. Devolver lo mismo en ambos casos convierte un problema de iluminación en un cliente bloqueado.

4. Estado actual de la implementación

Verificado contra la documentación oficial, campo por campo. **La integración con Facephi es correcta:** la URL, los campos del request, los valores de `method` y los nueve campos de respuesta coinciden.

4.1 Mapa de archivos

Archivo	Qué hace hoy
<code>src/micro-person-api/Endpoints/FacePhi.cs</code>	Expone <code>POST /facephi/validate</code> , delega al comando y devuelve <code>Results.Ok</code>
<code>src/micro-person-api/Application/Facephi/Commands/ValidateIdentityV2Cmd.cs</code>	Comando, validador <code>FluentValidation</code> y handler <code>MediatR</code>
<code>src/micro-person-api/Common/Interfaces/Biometric/IFacePhiService.cs</code>	Contrato del cliente de Facephi
<code>src/micro-person-api/Services/Biometric/FacePhiService.cs</code>	<code>EvaluatePassiveLivenessToken</code> : arma el payload y llama a <code>/onboarding/v2/identity</code>
<code>src/micro-person-api/Common/Models/Biometric/PassiveLivenessResults.cs</code>	<code>PassiveLivenessResult</code> : los 9 campos de la respuesta
<code>src/micro-person-api/Common/Enums/FacePhi/</code>	<code>FacephiLivenessResult</code>
<code>tests/UnitTests/Application/SoftTokenFacephi/ValidateIdentityV2CmdTests.cs</code>	Tests del validador y del handler

4.2 Lo que está bien y no hay que tocar

- La ruta `/facephi/validate` es genérica, sin semántica de Soft Token.
- El payload omite el objeto `tracking` completo cuando no hay datos, en vez de mandarlo con nulos.
- Los `LogInformation` registran método, códigos y `transactionId`, **nunca** los tokens biométricos.
- El validador exige `token1`, `bestImageToken` y `method`, y restringe `method` a `"3"` o `"5"`.
- Los tests verifican el mapeo campo por campo con `Verify`.

4.3 El problema de fondo

El handler devuelve el `PassiveLivenessResult` tal cual y el endpoint responde `Results.Ok(result)` **siempre**. Un rechazo biométrico —rostro que no coincide, ausencia de prueba de vida— llega al consumidor como HTTP 200. APP-5223 no tiene forma de saber si debe activar el token o no. Eso es lo que resuelve la sección 5.

5. Guía de cambios, archivo por archivo

5.0 Resumen

#	Archivo	Acción	Prioridad
1	Common/Enums/FacePhi/ValidationOutcome.cs	Nuevo	Alta
2	Common/Models/Biometric/ValidateIdentityV2Response.cs	Nuevo	Alta
3	Common/Exceptions/FacephiIntegrationException.cs	Nuevo (si no existe equivalente)	Alta
4	Application/Facephi/Commands/ValidateIdentityV2Cmd.cs	Modificar	Alta
5	Endpoints/FacePhi.cs	Modificar	Alta
6	Services/Biometric/FacePhiService.cs	Modificar	Alta
7	La clase que implementa <code>ExceptionHandler</code>	Modificar	Alta
8	Common/Models/Biometric/PassiveLivenessResults.cs	Modificar	Media
9	tests/UnitTests/Application/SoftTokenFacephi/	Renombrar + añadir tests	Media

Todas las rutas son relativas a `src/micro-person-api/` salvo las de `tests/`.

5.1 Nuevo · Common/Enums/FacePhi/ValidationOutcome.cs

Por qué: hoy no existe ningún tipo que represente el veredicto. Sin él, la decisión queda repartida entre el handler y el endpoint.

Dónde: misma carpeta donde ya vive `FacephiLivenessResult` — el `using onboarding_micro_person.Common.Enums.FacePhi;` de `PassiveLivenessResults.cs` confirma la ruta.

Archivo completo:

```
namespace onboarding_micro_person.Common.Enums.FacePhi;

/// <summary>
/// Veredicto de la validación biométrica de Identity Validation V2.
/// </summary>
public enum ValidationOutcome
{
    /// <summary>Rostro POSITIVE y prueba de vida Live. Continuar con la activación.</summary>
    Approved,

    /// <summary>El rostro no coincide o no se detectó vida. Cortar el flujo.</summary>
    Rejected,

    /// <summary>La captura no permitió evaluar. Pedir al cliente que repita.</summary>
    Retry,

    /// <summary>Falló Facephi o nuestra integración.</summary>
    Error
}
```

5.2 Nuevo · `Common/Models/Biometric/ValidateIdentityV2Response.cs`

Por qué: el consumidor necesita el veredicto explícito. Si devolvemos el `PassiveLivenessResult` crudo, cada consumidor tendría que reimplementar las tablas de la sección 3 — y tarde o temprano una de esas implementaciones se equivocará.

Dónde: junto a `PassiveLivenessResults.cs`, en `Common/Models/Biometric/`.

Archivo completo:

```

using onboarding_micro_person.Common.Enums.FacePhi;

namespace onboarding_micro_person.Common.Models.Biometric;

/// <summary>
/// Respuesta de POST /facephi/validate. Expone el veredicto ya interpretado
/// y los códigos crudos de Facephi para trazabilidad y soporte.
/// </summary>
public class ValidateIdentityV2Response
{
    /// <summary>Approved | Rejected | Retry | Error</summary>
    public ValidationOutcome Outcome { get; set; }

    /// <summary>Atajo para el consumidor: true sólo cuando Outcome es Approved.</summary>
    public bool Approved => Outcome == ValidationOutcome.Approved;

    /// <summary>Código de coincidencia facial devuelto por Facephi. Ver tabla 3.2.</summary>
    public int FacialAuthenticationResult { get; set; }

    /// <summary>Código de prueba de vida devuelto por Facephi. Ver tabla 3.3.</summary>
    public int PassiveLivenessResult { get; set; }

    /// <summary>Similitud facial. 1.0 = 100 %.</summary>
    public double FacialAuthenticationSimilarity { get; set; }

    /// <summary>Identificador de la transacción en Facephi. Para soporte.</summary>
    public string ServiceTransactionId { get; set; } = string.Empty;
}

```

Serializa el enum como texto ("Approved" , no 0) para que el JSON sea legible. Si el micro no tiene configurado `JsonStringEnumConverter` de forma global, añade `[JsonConverter(typeof(JsonStringEnumConverter))]` sobre la propiedad `Outcome` .

5.3 Nuevo • `Common/Exceptions/FacephiIntegrationException.cs`

Por qué: hoy, si Facephi falla, el `catch` de `FacePhiService` no tiene a dónde escalar el problema. Sin una excepción propia no hay forma de distinguir "Facephi se cayó" de "el cliente fue rechazado".

Dónde: primero busca si el micro ya tiene una carpeta de excepciones:

```
grep -rn "class .*Exception" --include=*.cs src/ | head
```

Si existe (por ejemplo `Common/Exceptions/`), añade el archivo ahí. Si no, créala.

Archivo completo:

```

namespace onboarding_micro_person.Common.Exceptions;

/// <summary>
/// Fallo comunicándose con Facephi: error HTTP, timeout o respuesta ilegible.
/// NO se usa cuando Facephi responde correctamente rechazando al cliente:
/// eso es un resultado de negocio, no una excepción.
/// </summary>
public class FacephiIntegrationException : Exception
{
    /// <summary>Código HTTP devuelto por Facephi, si lo hubo.</summary>
    public int? UpstreamStatusCode { get; }

    public FacephiIntegrationException(
        string message, int? upstreamStatusCode = null, Exception? innerException = null)
        : base(message, innerException)
    => UpstreamStatusCode = upstreamStatusCode;
}

```

5.4 Modificar · Application/Facephi/Commands/ValidateIdentityV2Cmd.cs

Tres cambios en este archivo. El comando y el validador **no se tocan**.

Cambio A — el tipo de retorno del comando

Línea a cambiar: la declaración de la clase `ValidateIdentityV2Cmd`.

```

// ANTES
public class ValidateIdentityV2Cmd : IRequest<PassiveLivenessResult>

// DESPUÉS
public class ValidateIdentityV2Cmd : IRequest<ValidateIdentityV2Response>

```

Cambio B — la firma del handler

Líneas a cambiar: la declaración de `ValidateIdentityV2CmdHandler` y la de `Handle`.

```
// ANTES
public class ValidateIdentityV2CmdHandler
    : IRequestHandler<ValidateIdentityV2Cmd, PassiveLivenessResult>
{
    public async Task<PassiveLivenessResult> Handle(
        ValidateIdentityV2Cmd request, CancellationToken cancellationToken)

// DESPUÉS
public class ValidateIdentityV2CmdHandler
    : IRequestHandler<ValidateIdentityV2Cmd, ValidateIdentityV2Response>
{
    public async Task<ValidateIdentityV2Response> Handle(
        ValidateIdentityV2Cmd request, CancellationToken cancellationToken)
```

Cambio C — evaluar el resultado antes de devolverlo

Dónde: al final del método `Handle`, **después** del segundo `_logger.LogInformation` (el que registra `ServiceResultCode`, `FacialResult`, `Similarity`, `LivenessResult` y `TransactionId`). Ahí hoy se devuelve `result` directamente; eso es lo que se reemplaza.

```

// ... el LogInformation de "Identity Validation V2 completed" se mantiene igual ...

var outcome = Evaluate(result);

_logger.LogInformation(
    "Identity Validation V2 outcome={Outcome}. TransactionId={TransactionId}",
    outcome, result.ServiceTransactionId);

return new ValidateIdentityV2Response
{
    Outcome = outcome,
    FacialAuthenticationResult = result.facialAuthenticationResult,
    PassiveLivenessResult = (int)result.passiveLivenessResult,
    FacialAuthenticationSimilarity = result.facialAuthenticationSimilarity,
    ServiceTransactionId = result.ServiceTransactionId
};
}

/// <summary>
/// Traduce la respuesta de Facephi a un veredicto. Ver secciones 3.2 y 3.3
/// del documento de APP-5258.
/// </summary>
private static ValidationOutcome Evaluate(PassiveLivenessResult r)
{
    // serviceResultCode == 0 sólo indica que el módulo se ejecutó,
    // NO que la persona haya sido aprobada.
    if (r.ServiceResultCode != 0)
        return ValidationOutcome.Error;

    var liveness = (int)r.passiveLivenessResult;

    // Fallos del motor de Facephi: no es culpa de la captura del cliente.
    // 10 = NoneBecauseInternalError, 15 = NoneBecauseLicenseError
    if (liveness is 10 or 15)
        return ValidationOutcome.Error;

    // Rechazos reales: el rostro no coincide (1 = NEGATIVE),
    // o no se detectó vida (17 = NoLive, 1 = Spoof deprecado).
    if (r.facialAuthenticationResult == 1 || liveness is 17 or 1)
        return ValidationOutcome.Rejected;

    // Lista blanca: sólo se aprueba con POSITIVE (3) y Live (3) explícitos.
    if (r.facialAuthenticationResult == 3 && liveness == 3)
        return ValidationOutcome.Approved;

    // Todo lo demás: la captura no permitió evaluar. El cliente repite.
    return ValidationOutcome.Retry;
}

```

Usings a añadir al inicio del archivo, si no están:

```
using onboarding_micro_person.Common.Enums.FacePhi;
```

Lista blanca, nunca lista negra. No uses `liveness != 17` para concluir que hay vida: la documentación de `evaluatePassiveLiveness` muestra un rechazo `NoLive` que devuelve `0`, no `17`. Sólo el `3` aprueba.

Verifica el enum `FacePhiLivenessResult` en `Common/Enums/FacePhi/`: el `(int)` de arriba asume que sus miembros tienen asignados los valores de la tabla 3.3 (`Live = 3`, `NoLive = 17`). Si el enum se declaró sin valores explícitos, C# numera desde 0 en orden de declaración y el cast daría números equivocados. Es una verificación de dos minutos que evita un fallo silencioso.

5.5 Modificar `Endpoints/FacePhi.cs`

Dos cambios: el tipo declarado en `Produces` y el mapeo del veredicto a código HTTP.

Cambio A — `Produces`

Dónde: dentro de `Map(WebApplication app)`, en la cadena del `group.MapPost("/validate", ...)`.

```
// ANTES
.Produces<PassiveLivenessResult>(StatusCodes.Status200OK)
.Produces(StatusCodes.Status400BadRequest)
.Produces(StatusCodes.Status401Unauthorized)
.Produces(StatusCodes.Status403Forbidden)
.Produces(StatusCodes.Status502BadGateway)
.Produces(StatusCodes.Status504GatewayTimeout)

// DESPUÉS
.Produces<ValidateIdentityV2Response>(StatusCodes.Status200OK)
.Produces<ValidateIdentityV2Response>(StatusCodes.Status422UnprocessableEntity)
.Produces(StatusCodes.Status400BadRequest)
.Produces(StatusCodes.Status401Unauthorized)
.Produces(StatusCodes.Status403Forbidden)
.Produces(StatusCodes.Status502BadGateway)
.Produces(StatusCodes.Status504GatewayTimeout)
```

Cambio B — el método `ValidateIdentityV2`

Dónde: el método que hoy hace `return Results.Ok(result);`, al final del archivo.

```
// ANTES
public async Task<IResult> ValidateIdentityV2(
    ValidateIdentityV2Cmd request,
    ISender sender,
    CancellationToken cancellationToken)
{
    var result = await sender.Send(request, cancellationToken);
    return Results.Ok(result);
}

// DESPUÉS
public async Task<IResult> ValidateIdentityV2(
    ValidateIdentityV2Cmd request,
    ISender sender,
    CancellationToken cancellationToken)
{
    var result = await sender.Send(request, cancellationToken);

    // Rejected y Retry NO son errores del servicio: son resultados de negocio.
    // Por eso 422 con cuerpo, y no 400 ni 500.
    return result.Outcome switch
    {
        ValidationOutcome.Approved => Results.Ok(result),
        ValidationOutcome.Error    => Results.Json(
            result, statusCode: StatusCodes.Status502BadGateway),
        _                          => Results.Json(
            result, statusCode: StatusCodes.Status422UnprocessableEntity)
    };
}
```

Usings a añadir:

```
using onboarding_micro_person.Common.Enums.FacePhi;
using onboarding_micro_person.Common.Models.Biometric;
```

Contrato resultante

Outcome	HTTP	Qué hace APP-5223
Approved	200	Continúa con la activación del Soft Token
Rejected	422	Corta el flujo. No pedir recaptura
Retry	422	Pide al cliente repetir la captura
Error	502	Falla técnica, reintentable a nivel de servicio

El consumidor distingue `Rejected` de `Retry` leyendo el campo `outcome` del cuerpo. Ambos son 422 porque en los dos casos la petición era válida pero no se pudo completar la operación.

Alternativa mínima, si cambiar la forma del cuerpo obliga a coordinar con quien ya está consumiendo: mantener `PassiveLivenessResult` como respuesta y cambiar **sólo el código HTTP**. Es menos claro, pero resuelve el problema de fondo. Lo que **no** es aceptable es seguir devolviendo `200` para todo.

5.6 Modificar · `Services/Biometric/FacePhiService.cs`

Cuatro cambios dentro del método `EvaluatePassiveLivenessToken`. Este archivo es largo; los anclajes indican el punto exacto.

Cambio A — headers obligatorios

Dónde: justo después de construir el `HttpRequestMessage`, antes de enviarlo. En el código revisado eso es:

```
var httpRequest = new HttpRequestMessage(HttpMethod.Post, url)
{
    Content = new StringContent(json, Encoding.UTF8, "application/json")
};
```

Añadir inmediatamente debajo:

```
// x-api-key es obligatorio. family es requerido al enviar el objeto tracking,
// y este payload lo incluye cuando el SDK lo entrega.
httpRequest.Headers.Add("x-api-key", _appSettings.FACEPHI_IDENTITY_API_KEY);
httpRequest.Headers.Add("family", "OnBoarding");
```

Antes de dar esto por hecho, verifica que la llave exista en configuración:

```
grep -rn "FACEPHI_IDENTITY" --include=*.cs --include=*.json src/
```

Si el micro ya llamaba a otros endpoints de Identity API, la llave probablemente ya está y sólo hay que usarla. Si no existe, añádela junto a `FACEPHI_IDENTITY_API_BASE_URL` en la clase de settings y en la configuración — **el valor no se versiona**: va por variable de entorno o vault.

Cambio B — deserialización explícita

Dónde: donde el método convierte el cuerpo de la respuesta en `PassiveLivenessResult`.

```
// Si el JSON llega en camelCase y el modelo tiene propiedades en PascalCase,
// sin esta opción los campos quedan en 0 sin lanzar ningún error.
var options = new JsonSerializerOptions { PropertyNameCaseInsensitive = true };
var result = JsonSerializer.Deserialize<PassiveLivenessResult>(body, options);

if (result is null)
    throw new FacephiIntegrationException(
        "Facephi devolvió un cuerpo vacío o no interpretable.");
```

El cambio 5.8 hace esto redundante al poner `[JsonPropertyName]` explícito; aplicar ambos no hace daño y protege si alguien añade un campo nuevo sin atributo.

Cambio C — no tragarse los errores HTTP

Dónde: después de enviar la petición, antes de leer el cuerpo.

```
if (!response.IsSuccessStatusCode)
{
    // No se loguea el cuerpo: puede contener datos biométricos.
    _logger.LogError(
        "Facephi respondió {StatusCode} en Identity Validation V2.",
        (int)response.StatusCode);

    throw new FacephiIntegrationException(
        $"Facephi respondió con código {(int)response.StatusCode}.",
        (int)response.StatusCode);
}
```

Cambio D — el `catch`

Dónde: el `catch` que cierra el `try` de este método.

El problema actual: si el `catch` registra el error y devuelve un `PassiveLivenessResult` vacío, ese objeto llega al handler con `ServiceResultCode = 0` y la regla de la sección 5.4 lo interpretaría como ejecución correcta con códigos en cero → `Retry`. Un Facephi caído se reportaría como "repite la captura".

```

catch (FacephiIntegrationException)
{
    throw;    // ya tiene el contexto correcto
}
catch (TaskCanceledException ex) when (!cancellationToken.IsCancellationRequested)
{
    throw new FacephiIntegrationException(
        "Timeout invocando Identity Validation V2.", 504, ex);
}
catch (HttpRequestException ex)
{
    throw new FacephiIntegrationException(
        "Error de red invocando Identity Validation V2.", 502, ex);
}

```

Ojo con el `catch` del `FinishTrackingAsync`, unas líneas más arriba en el mismo archivo: ese sí debe seguir tragándose la excepción con un `LogWarning`, porque el tracking es accesorio y no debe tumbar la validación. Son dos criterios distintos y conviven bien.

5.7 Modificar · la clase que implementa `ExceptionHandler`

Por qué: sin esto, la `FacephiIntegrationException` que ahora lanza el servicio sale como HTTP 500 genérico en vez de 502/504.

Dónde encontrarla: el proyecto usa la plantilla Clean Architecture (`EndpointGroupBase`, `MapGroup`, `ISender`), donde el manejador suele llamarse `CustomExceptionHandler` y vivir junto a `EndpointGroupBase` — es decir, probablemente en `Common/Infrastructure/`, por el `using onboarding_micro_person.Common.Infrastructure;` del archivo de endpoints. Confírmalo:

```

grep -rn "ExceptionHandler\|ProblemDetails" --include=*.cs src/

```

Qué añadir: una entrada más en el diccionario o `switch` de mapeo de excepciones.

```

{ typeof(FacephiIntegrationException), HandleFacephiIntegrationException },

```

```
private async Task HandleFacephiIntegrationException(
    HttpContext httpContext, Exception ex)
{
    var exception = (FacephiIntegrationException)ex;

    // 504 sólo si Facephi realmente dio timeout; el resto es 502.
    httpContext.Response.StatusCode = exception.UpstreamStatusCode == 504
        ? StatusCodes.Status504GatewayTimeout
        : StatusCodes.Status502BadGateway;

    await httpContext.Response.WriteAsJsonAsync(new ProblemDetails
    {
        Status = httpContext.Response.StatusCode,
        Title = "Error de integración con Facephi",
        // Sin detalles internos hacia el cliente.
        Detail = "No fue posible completar la validación biométrica."
    });
}
```

Verifica también que el pipeline tenga registrado el `ValidationBehaviour` de `FluentValidation`: es lo que convierte un `method = "4"` en un `400` con los mensajes del validador. Si el `400` no sale, es que ese comportamiento no está en el pipeline de MediatR.

5.8 Modificar · `Common/Models/Biometric/PassiveLivenessResults.cs`

Por qué: los nombres de los nueve campos son **correctos** para Identity V2, pero cuatro están en PascalCase y sólo bindean si la deserialización es *case-insensitive*. Hacerlo explícito elimina esa dependencia.

Archivo completo resultante:

```

using System.Text.Json.Serialization;
using onboarding_micro_person.Common.Enums.FacePhi;

namespace onboarding_micro_person.Common.Models.Biometric;

/// <summary>
/// Respuesta de POST /onboarding/v2/identity (Identity Validation V2).
/// Los nombres JSON son los de ESE endpoint: otros endpoints de Facephi
/// usan el prefijo "service" (serviceLivenessResult, etc.).
/// </summary>
public class PassiveLivenessResult
{
    [JsonPropertyName("serviceTransactionId")]
    public string ServiceTransactionId { get; set; } = string.Empty;

    [JsonPropertyName("serviceResultCode")]
    public int ServiceResultCode { get; set; }

    [JsonPropertyName("serviceResultLog")]
    public string ServiceResultLog { get; set; } = string.Empty;

    [JsonPropertyName("serviceTime")]
    public string ServiceTime { get; set; } = string.Empty;

    [JsonPropertyName("facialAuthenticationResult")]
    public int facialAuthenticationResult { get; set; }

    [JsonPropertyName("facialAuthenticationLog")]
    public string facialAuthenticationLog { get; set; } = string.Empty;

    [JsonPropertyName("facialAuthenticationSimilarity")]
    public double facialAuthenticationSimilarity { get; set; }

    [JsonPropertyName("passiveLivenessResult")]
    public FacephiLivenessResult passiveLivenessResult { get; set; }

    [JsonPropertyName("passiveLivenessLog")]
    public string passiveLivenessLog { get; set; } = string.Empty;
}

```

Riesgo heredado a revisar

Este modelo se usa también en el flujo de onboarding que llama a `/services/evaluatePassiveLiveness`. **Ese endpoint devuelve `serviceLivenessResult`, no `passiveLivenessResult`**. Si ambos flujos deserializan en la misma clase, el resultado de vida del flujo viejo estaría cayendo en `0` (None) de forma silenciosa — y con los `[JsonPropertyName]` de arriba el problema se vuelve permanente en vez de accidental.

```
grep -rn "PassiveLivenessResult" --include=*.cs src/
```

Si aparece en más de un flujo, separa en dos modelos: `IdentityValidationV2Result` para este endpoint y `PassiveLivenessResult` para el otro.

5.9 Modificar · tests

Cambio A — renombrar la carpeta

```
tests/UnitTests/Application/SoftTokenFacephi/ → tests/UnitTests/Application/Facephi/
```

El namespace de dentro ya dice `onboarding_micro_person.Tests.UnitTests.Application.Facephi`, así que sólo hay que mover la carpeta. Es exactamente el nombre que el equipo pidió no usar.

Cambio B — actualizar el test del handler

El test `Handler_MapsRequest_AndReturnsResult` **dejará de compilar** al cambiar el tipo de retorno: hoy hace `Assert.That(result.ServiceResultCode, Is.EqualTo(0))` sobre lo que ahora es un `ValidateIdentityV2Response`.

```
// ANTES
Assert.That(result.ServiceResultCode, Is.EqualTo(0));
Assert.That(result.facialAuthenticationResult, Is.EqualTo(3));
Assert.That(result.passiveLivenessResult, Is.EqualTo(FacephiLivenessResult.Live));

// DESPUÉS
Assert.That(result.Outcome, Is.EqualTo(ValidationOutcome.Approved));
Assert.That(result.Approved, Is.True);
Assert.That(result.FacialAuthenticationResult, Is.EqualTo(3));
Assert.That(result.PassiveLivenessResult, Is.EqualTo(3));
```

El `_facePhiMock.Verify(...)` que comprueba el mapeo del request **no cambia**: sigue siendo válido y es la parte más valiosa de ese test.

Cambio C — tests de la regla de decisión

El veredicto es la lógica nueva y hoy no tiene cobertura. Un `TestCase` por fila cubre toda la tabla:

```

[TestCase(3, 3, ValidationOutcome.Approved)] // POSITIVE + Live
[TestCase(1, 3, ValidationOutcome.Rejected)] // NEGATIVE: no es la misma persona
[TestCase(3, 17, ValidationOutcome.Rejected)] // NoLive: sin prueba de vida
[TestCase(3, 0, ValidationOutcome.Retry)] // None: no se pudo evaluar
[TestCase(3, 4, ValidationOutcome.Retry)] // mala calidad de imagen
[TestCase(3, 18, ValidationOutcome.Retry)] // ojos cerrados
[TestCase(0, 3, ValidationOutcome.Retry)] // no se pudo verificar el rostro
[TestCase(3, 10, ValidationOutcome.Error)] // error interno de Facephi
[TestCase(3, 15, ValidationOutcome.Error)] // error de licencia
public async Task Handler_Evalua_El_Verdicto_Segun_Los_Codigos_De_Facephi(
    int facialResult, int livenessResult, ValidationOutcome esperado)
{
    _facePhiMock
        .Setup(x => x.EvaluatePassiveLivenessToken(
            It.IsAny<PassiveLivenessRequest>(), It.IsAny<CancellationToken>()))
        .ReturnsAsync(new PassiveLivenessResult
        {
            ServiceResultCode = 0,
            facialAuthenticationResult = facialResult,
            passiveLivenessResult = (FacephiLivenessResult)livenessResult
        });

    var result = await _handler.Handle(
        new ValidateIdentityV2Cmd
        {
            Token1 = "reference-token",
            BestImageToken = "best-image-token",
            Method = "3"
        },
        CancellationToken.None);

    Assert.That(result.Outcome, Is.EqualTo(esperado));
}

```

Cambio D — test de deserialización

El test actual construye el `PassiveLivenessResult` en memoria, así que nunca ejercita el binding del JSON: si los nombres estuvieran mal, no se enteraría.

```
[Test]
public void Deserializa_La_Respuesta_De_Ejemplo_De_Facephi()
{
    const string json = """
    {
        "serviceTransactionId": "2db602ee-3564-4304-af95-92a52eaae12d",
        "serviceResultCode": 0,
        "serviceResultLog": "[identity] Service executed ok",
        "serviceTime": "2235",
        "facialAuthenticationResult": 3,
        "facialAuthenticationLog": "Positive",
        "facialAuthenticationSimilarity": 0.99214232,
        "passiveLivenessResult": 3,
        "passiveLivenessLog": "Live"
    }
    """;

    var result = JsonSerializer.Deserialize<PassiveLivenessResult>(json!);

    Assert.Multiple(() =>
    {
        Assert.That(result.ServiceResultCode, Is.EqualTo(0));
        Assert.That(result.ServiceTransactionId, Is.Not.Empty);    // detecta binding roto
        Assert.That(result.ServiceTime, Is.EqualTo("2235"));
        Assert.That(result.facialAuthenticationResult, Is.EqualTo(3));
        Assert.That((int)result.passiveLivenessResult, Is.EqualTo(3));
    });
}
```


Cambio E — tests negativos del validador

```
[TestCase("4")]
[TestCase("1")]
[TestCase("")]
public void Validator_Rechaza_Methods_No_Soportados(string method)
{
    var result = new ValidateIdentityV2CmdValidator().Validate(
        new ValidateIdentityV2Cmd
        {
            Token1 = "t",
            BestImageToken = "b",
            Method = method
        });

    Assert.That(result.IsValid, Is.False);
}

[Test]
public void Validator_Rechaza_OperationId_Que_No_Es_Uuid()
{
    var result = new ValidateIdentityV2CmdValidator().Validate(
        new ValidateIdentityV2Cmd
        {
            Token1 = "t",
            BestImageToken = "b",
            Method = "3",
            Tracking = new FacephiTrackingExtraData { OperationId = "no-es-uuid" }
        });

    Assert.That(result.IsValid, Is.False);
}
```

5.10 Renombres pendientes (opcional, baja prioridad)

No afectan al funcionamiento, pero evitan confusión futura. Hazlos con el *rename* del IDE, en un commit separado del resto:

Actual	Sugerido	Motivo
IFacePhiService.EvaluatePassiveLivenessToken	ValidateIdentityV2Async	Llama a Identity V2, no a evaluatePassiveLiveness , que es otro endpoint. Además el resto del servicio usa el sufijo Async
PassiveLivenessRequest	IdentityValidationV2Request	Mismo motivo
PassiveLivenessResult	IdentityValidationV2Result	Sólo si se confirma el riesgo de 5.8

Tocan IFacePhiService.cs , FacePhiService.cs , ValidateIdentityV2Cmd.cs y los tests.

6. Orden de aplicación y verificación

Este orden mantiene el proyecto compilando en cada paso.

Paso	Cambio	El proyecto compila
1	5.1 <code>ValidationOutcome</code>	Sí — sólo añade un enum
2	5.2 <code>ValidateIdentityV2Response</code>	Sí — sólo añade un modelo
3	5.3 <code>FacephiIntegrationException</code>	Sí — sólo añade una excepción
4	5.8 <code>[JsonPropertyName]</code>	Sí — no cambia firmas
5	5.4 comando y handler	No , hasta completar el paso 6
6	5.5 endpoint	Sí — se cierra el cambio de tipo
7	5.6 servicio: headers, errores, catch	Sí
8	5.7 manejador de excepciones	Sí
9	5.9 tests	Sí

Los pasos 5 y 6 van juntos: cambiar el tipo de retorno del comando rompe el endpoint hasta que se actualiza.

6.1 Verificación

```
dotnet build
dotnet test
```

```
# Aprobado
curl -sX POST https://localhost:5001/facephi/validate \
  -H "Content-Type: application/json" \
  -d '{"token1": "<base64-cedula>", "bestImageToken": "<token-selfie>", "method": "3"}'
# → 200 { "outcome": "Approved", "approved": true, ... }

# Con tracking
curl -sX POST https://localhost:5001/facephi/validate \
  -H "Content-Type: application/json" \
  -d '{"token1": "...", "bestImageToken": "...", "method": "5",
      "tracking": {"extraData": "...", "operationId": "123e4567-e89b-12d3-a456-426614174000"} }'

# method inválido → 400 del validador
curl -sX POST https://localhost:5001/facephi/validate \
  -H "Content-Type: application/json" \
  -d '{"token1": "x", "bestImageToken": "y", "method": "4"}'

# Sin bestImageToken → 400
curl -sX POST https://localhost:5001/facephi/validate \
  -H "Content-Type: application/json" \
  -d '{"token1": "x", "method": "3"}'
```

Con una URL de Facephi inválida en configuración, el servicio debe responder **502**, no 200 ni 500.

Y por último, la verificación que no puede faltar:

```
grep -ri "<el token que usaste en las pruebas>" logs/
# No debe devolver ninguna línea: sería una fuga de datos biométricos.
```

7. Checklist de cierre

#	Tarea	Sección	Archivo	Prioridad
1	Enum <code>ValidationOutcome</code>	5.1	<code>Common/Enums/FacePhi/ValidationOutcome.cs</code>	Alta
2	Modelo de respuesta	5.2	<code>Common/Models/Biometric/ValidateIdentityV2Response.cs</code>	Alta
3	Excepción de integración	5.3	<code>Common/Exceptions/FacephiIntegrationException.cs</code>	Alta
4	Regla de decisión en el handler	5.4	<code>Application/Facephi/Commands/ValidateIdentityV2Cmd.cs</code>	Alta
5	Mapeo a 200 / 422 / 502	5.5	<code>Endpoints/FacePhi.cs</code>	Alta
6	Headers <code>x-api-key</code> y <code>family</code>	5.6 A	<code>Services/Biometric/FacePhiService.cs</code>	Alta
7	Errores HTTP y <code>catch</code> sin objetos vacíos	5.6 C-D	<code>Services/Biometric/FacePhiService.cs</code>	Alta
8	Traducción de la excepción a 502/504	5.7	manejador de <code>ExceptionHandler</code>	Alta
9	Verificar los valores del enum <code>FacephiLivenessResult</code>	5.4	<code>Common/Enums/FacePhi/</code>	Alta
10	<code>[JsonPropertyName]</code> explícito	5.8	<code>Common/Models/Biometric/PassiveLivenessResults.cs</code>	Media
11	Revisar el modelo compartido con <code>evaluatePassiveLiveness</code>	5.8	—	Media
12	Tests del veredicto, deserialización y negativos	5.9	<code>tests/UnitTests/Application/Facephi/</code>	Media
13	Renombrar la carpeta <code>SoftTokenFacephi</code>	5.9 A	<code>tests/UnitTests/Application/</code>	Media
14	Renombres del servicio y los modelos	5.10	varios	Baja
15	Corregir el modelo de datos de APP-5258: <code>token2</code> no aplica	1	Jira	Baja

8. Preguntas abiertas

#	Pregunta	Para quién
1	¿Qué <code>method</code> usará la app: <code>"3"</code> con imagen abierta o <code>"5"</code> con widget SelphID? Define de dónde sale <code>token1</code>	Producto / Móvil
2	¿El endpoint <code>/facephi/validate</code> lleva autenticación, o queda interno tras el gateway?	Arquitectura / Seguridad
3	¿Cuántos reintentos se permiten ante <code>Retry</code> antes de bloquear la activación?	Producto / Riesgo
4	Ante <code>Rejected</code> , ¿APP-5223 bloquea, alerta a fraude, o ambas?	Producto / Riesgo
5	Capitalización correcta del header <code>family</code> : <code>OnBoarding</code> u <code>Onboarding</code>	Facephi

Documento de la implementación real de APP-5258, contrastado campo por campo contra la documentación oficial de Facephi Identity API. El código de la sección 5 son cambios propuestos y no ha sido compilado.